PROJECT PRESENTATION

[SOLO ARC · SafeShell — Command Safety Engine]

[Track 1 — AI at Application Level] · [Ref Code]

01

Design Choice

Problem Statement

- The shell has no safety net: rm -rf /, dd of=/dev/sda or mkfs can destroy data in an instant. Users need a layer that understands command intent before execution, detects risky ops, explains impact, and suggests safer alternatives.

Design Approach

- Intercept every command pre-execution; classify with three cross-checking layers (deterministic rules, ML classifier, local on-device LLM) and hard-block destructive ops. Wrap risky commands in a transaction: snapshot files, simulate in a sandbox, keep an undo plan so damage always rolls back.

Reasoning

- Every layer can fail: rules give guaranteed hard-blocks, ML catches novel vectors, the LLM explains ambiguous ~0.65-confidence cases in plain language. Transactions turn detection into reversibility - users stay in control instead of losing data.

Architecture

Architecture Choices

- Three analysis layers (rule engine, ML, LLM) agree on a verdict. A SafeShell transaction layer then attaches to every WARN command: extracts target paths, snapshots content + metadata under ~/.csengine/tx/<id> (sha256-tagged trees), builds an undo plan (static + LLM). Simulation runs the command in a bubblewrap sandbox and diffs the filesystem before anything real executes.

Reasoning

- Modular design keeps the fast path (rules + ML, ~0.4 ms) on most traffic while the LLM reasons only about ambiguous commands. Transaction/simulation layers are purely additive - shell behaviour is unchanged, only safety is added. Offline-first design works on air-gapped systems like BOSS OS.

Technology Choices

Technologies Used

- Python 3.13 + scikit-learn GradientBoosting (Test Acc 0.8225, Macro-F1 0.7906). Ollama + Qwen 2.5-3B q4 for on-device explanations. bash DEBUG-trap preexec + zsh Enter-key widget for blocking hooks. bubblewrap sandbox, sha256+JSON transaction store, real-PTY end-to-end test harnesses.

Reasoning

- Fully on-device - no cloud API, matching the offline/sovereign security requirement. GradientBoosting wins on macro-F1 while the 27-rule engine keeps destructive-class recall competitive. Model-swappable Ollama backend; bwrap needs no kernel changes, so it runs on stock Ubuntu, Debian and BOSS OS.

Demo

Demo Walkthrough

- 1) rm -rf /, mkfs and dd-on-/dev/sda BLOCKED deterministically (exit 2, audit logged). 2) chmod -R 777 needs confirmation + shows a simulation diff, then a one-command undo. 3) An obfuscated hex/base64 command is decoded and the on-device model explains the hidden payload. 4) 17/17 verification script runs on camera.

Key Features

- 3-layer hybrid detection (rules + ML + local LLM) • transactional undo with AI-generated plans • sandbox simulation of exact impact • live ‘thinking’ notice • ~0.4 ms rule+ML path • fully offline • Ubuntu / Debian / BOSS OS.